TSP_SA_and_GA_Assignm...
File

I need help getting started on this assignemtn, dont provide answers just guide me. I dont know python well and I have never used jupyter notebooks like this

Thought for 1m 6s >

Got you. Let's treat this like "how do I drive this notebook + what am I supposed to fill in" without me writing the actual filled-in solutions.

1) How to use a Jupyter notebook (the minimum you need)

What a notebook is: a list of cells you run top-to-bottom. Some are **Markdown** (instructions) and some are **code**.

The 4 moves you'll use constantly

- **Run a cell:** click the cell, press **Shift+Enter**
- **Run and stay in cell: Ctrl+Enter**
- **Add a new cell below: B** (in command mode; press **Esc** first if you're typing)
- **Restart + run all:** Kernel menu → **Restart & Run All** (useful when things get weird)

If you get errors

- Read the *last line* of the traceback first (it usually says what's wrong).
- If you changed something earlier and now everything breaks: **Restart & Run All** to clear state.

Workflow tip: Run the notebook once *without changing anything* up to the first TODO so you know the provided utilities work.

---

2) What you actually need to implement (your TODOs)

In this notebook, the TODOs are basically:

1. **Simulated Annealing (SA)**: acceptance rule + cooling schedule
2. **Genetic Algorithm (GA)**: tournament selection + order crossover (OX) + swap mutation
3. **Comparison cell**: print/plot summary once both work

I'll guide you through each in the safest "implement → sanity check → integrate" order.

---

3) Part 5A — Simulated Annealing (SA): how to fill the two TODOs

You'll see code that repeatedly:

- creates a candidate tour using a **2-opt neighbor**
- computes `delta = cand_len - cur_len`
- decides whether to accept the candidate
- updates the temperature `T`

A) Acceptance rule TODO (the "should we move?" decision)

Think in two cases:

Case 1: Candidate is better

- `delta` is **negative or zero** (candidate length is shorter or same)
- The algorithm should accept these moves deterministically.

Case 2: Candidate is worse

- `delta` is **positive**
- SA sometimes accepts worse moves early on to escape local minima.
- The probability depends on **delta** and **temperature T**:
  - hotter T → more likely to accept bad moves
  - colder T → less likely

How to implement without guessing

- You'll want:
  - a random number `u` in `[0,1)` (look for `rng.random()`)
  - an acceptance probability computed from `delta` and `T`
- Then: accept if `u < probability`.

Sanity checks you should do after you implement

- Temporarily print a few lines every N iterations:
  - current `T`, `delta`, whether accepted
- You should observe:
  - early iterations: occasional acceptance of worse moves
  - later iterations: almost none

B) Cooling schedule TODO (the "how does T change?" part)

Most SA implementations use a schedule like "multiply by something slightly less than 1 each step."

You'll see config variables like `t0` and `alpha`. That's your hint:

- start temperature at `t0`
- update rule uses `alpha` each iteration

Sanity checks

- Print T at iteration 0, 1000, 5000, etc.
- It should steadily decrease (not increase, not become negative).
- If it becomes 0 too early, SA will freeze and stop accepting worse moves.

---

4) Part 5B — Genetic Algorithm (GA): implement the three building blocks first

In GA, it helps to implement in this order:

1. `mutate_swap`
2. `tournament_select`
3. `order_crossover_ox`

Because mutation is easiest, and crossover is the most error-prone.

A) `mutate_swap(tour)`

A **tour** is a permutation of city indices.

Swap mutation means:

- choose two different positions `i` and `j`
- swap the cities at those positions

- return the mutated tour (usually copy so you don't mutate the original unexpectedly)

Sanity checks

- The result should contain the *same elements* as before.
- Length should be the same.
- Exactly two positions should differ.

A quick self-check idea:

- `sorted(new_tour) == sorted(old_tour)` should be True
- (This doesn't solve it for you; it just verifies "still a permutation.")

B) `tournament_select(pop, lengths, k)`

You're minimizing tour length.
Tournament selection means:

- randomly pick `k` individuals (indices) from the population
- choose the one with the **best** fitness (here: *smallest* length)
- return that tour

Sanity checks

- If `k=1`, selection should be basically random.
- If `k` is large (like near pop size), selection should almost always pick the current best.

C) `order_crossover_ox(parent1, parent2)`

OX is the one that trips people up. Implement it carefully and test it in isolation.

High-level OX idea:

1. Pick a random slice `[a:b]` from parent1 and copy it into the child at the same positions.
2. Starting from position `b` in parent2 (wrapping around), take cities **in order** that are *not already in the child*, and fill the remaining empty slots in the child (wrapping around).

Sanity checks (super important)

- Child must be a valid permutation:
  - contains every city exactly once
- Child should preserve:
  - the copied slice from parent1 exactly

- relative order of remaining items as they appear in parent2

Debugging tip

- Print parents, chosen cut points, and the intermediate child with “blanks” for empty spots.

---

5) How to test each function without “running the whole GA/SA”

This is the biggest Jupyter advantage: you can make tiny test cells.

Add a small code cell under your function and do micro-tests like:

- create a tiny tour `[0,1,2,3,4,5]`
- run mutation/crossover a few times
- check permutation validity

If something fails:

- fix the small function first
- only then run the big `genetic_algorithm_tsp(...)` loop

---

6) The final TODO (comparison cell)

Once both algorithms run:

- you’ll likely have histories like `sa_hist` and `ga_hist`
- the notebook hints at plotting + printing a summary

Do this last. Your goal is just:

- confirm both improve over baseline
- visualize convergence curves

---

7) If you tell me what’s blocking you, I’ll guide the next step

You don’t need to paste the whole notebook—just answer one of these:

- Are you stuck on **Jupyter mechanics** (running cells, errors, etc.)?
- Or stuck on **SA acceptance/cooling** conceptually?
- Or stuck on **OX crossover** (most common)?

If you paste the **error message** (just the traceback text), I'll help you interpret it without giving away the filled-in code.

I have never worked with SA yet so this is my introduction so i am intemidated and dont know where to start

Totally normal. SA feels spooky until you realize it's basically **"hill climbing + occasional intentional mistakes that fade over time."** Here's the easiest way to start without getting overwhelmed.

The mental model (in plain English)

You're trying to minimize tour length.

At each step you:

1. **Make a small change** to your current tour (a neighbor move, like 2-opt).
2. **Measure the change in cost**:

$$\Delta = \text{new_length} - \text{current_length}$$

3. Decide:

- If the move is **better** ($\Delta \leq 0$): **take it**
- If the move is **worse** ($\Delta > 0$): **maybe take it**, with probability that's higher when:
  - Δ is small (only a little worse)
  - temperature **T** is high (early in the run)

4. **Cool down T** so you become pickier over time.

That's it.

Where to start in your notebook (no coding yet)

Step 1: Find the SA function cell

Scroll until you see something like:

- `simulated_annealing_tsp(...)`
- or a loop with variables like `T`, `t0`, `alpha`, `iters`, `delta`

Don't touch anything yet. Just identify:

- Where `delta` is computed
- Where the "accept move?" decision happens (likely a TODO)
- Where temperature is updated (another TODO)

Step 2: Run everything above SA

In Jupyter: **Shift+Enter** through cells above until you reach SA.
Goal: no errors, and you see imports/utilities loaded.

If something errors before SA, fix that first.

The only two SA pieces you'll implement

A) The acceptance decision

You will literally write the logic for:

- **If Δ ≤ 0**: accept
- **Else**: accept with probability based on `Δ` and `T`

A good way to think about the probability:

- If it's *slightly worse* and T is *hot*: "sure, why not"
- If it's *much worse* or T is *cold*: "nope"

(Your skeleton likely hints at the exact formula; you'll fill it in there.)

B) The cooling schedule

You'll update `T` each iteration so it decreases smoothly.

(Again, the skeleton usually gives variables like `alpha` or `cooling_rate` —you'll use those.)

A super-safe way to implement SA (so you don't feel lost)

Phase 1: Make it work with "always accept better, never accept worse"

Before you even do probabilistic acceptance, temporarily implement:

- accept if $\Delta \le 0$
- reject otherwise

This is basically hill climbing. It will run and you can confirm your loop is working.

What you should observe: improvements happen, then it gets stuck (plateau).

Phase 2: Turn on "sometimes accept worse"

Now add the probabilistic part for $\Delta > 0$.

What you should observe: it escapes plateaus earlier; later it stabilizes.

Phase 3: Tune cooling (if required)

If it accepts too many bad moves too long → cool faster
If it gets stuck too early → cool slower / start hotter

You usually don't need heavy tuning for an assignment; just get the intended schedule correct.

What to watch for (quick intuition checks)

If your SA is behaving strangely, it's usually one of these:

- **T goes negative or hits 0 immediately** → cooling wrong
- **It accepts worse moves even at the end** → T not cooling or probability wrong
- **It never accepts worse moves** → probability always ~0 (T too small or sign error)
- **Tour stops being a permutation** → neighbor function is broken (less likely if provided)

Your next step right now (do this, then tell me what you see)

1. Open the SA cell and locate the two TODOs.
2. Tell me the **variable names** you see around them (just the names, not your code), like:

- `T`, `t0`, `alpha`, `delta`, `rng`, `iters`, etc.

Then I'll tell you *exactly* how to plug your logic into that skeleton in a way that matches the notebook's structure—still without giving the final filled-in "answer code."

```
if delta <= 0:
    accept = True
else:
    accept = math.exp(-delta / T) > rng.random()
```

Nice — that's exactly the *shape* of the SA acceptance rule you want.

Here's how to understand it (and a couple of practical "don't get bitten" checks), without changing your logic.

What your code is doing

- `delta <= 0` → the candidate is **better or equal**, so you **always accept**.
- otherwise:
  - `math.exp(-delta / T)` computes an **acceptance probability** between 0 and 1
    - bigger `delta` (worse move) → probability smaller
    - bigger `T` (hotter) → probability larger
  - `> rng.random()` does the coin flip:
    - `rng.random()` is uniform in `[0, 1)`
    - accept when the probability beats that random number

That's SA in one line.

Quick sanity checks (super worth doing)

1) Make sure `T` never hits 0

If `T == 0`, `-delta / T` will crash (division by zero) or blow up.
So:

- start `T` at something positive
- cool it but don't let it reach 0 (or clamp with a tiny minimum if the skeleton suggests it)

2) Make sure `delta` is the right sign

This rule assumes:

- `delta = new_length - current_length`

If you accidentally computed it the other way around, you'd accept worse moves too often or reject good moves.

3) Spot-check behavior early vs late

Add a temporary print every, say, 500 iterations:

- `T`
- `delta`
- `accept`

You should see:

- early (high T): some accepts even when `delta > 0`
- late (low T): almost none

Tiny style note (optional)

Some skeletons prefer:

- compute `p = exp(-delta/T)`
- then `accept = (p > rng.random())`

But your one-liner is perfectly fine.

If you paste (or describe) your **temperature update line** right after this (the cooling schedule), I can help you sanity-check that too—cooling is the other place SA usually goes off the rails.

```
T = cfg.t0

    for it in range(cfg.iters):
        cand_tour = random_two_opt_neighbor(cur_tour, rng)
        cand_len = tour_length(cities, cand_tour)
        delta = cand_len - cur_len

        accept = False
        # TODO: acceptance rule
```

```
        # Is accept True or false when delta <= 0 or delta >0?
        if delta <= 0:
            accept = True
        else:
            accept = math.exp(-delta / T) > rng.random()



        if accept:
            cur_tour, cur_len = cand_tour, cand_len

        if cur_len < best_len:
            best_tour, best_len = cur_tour[:], cur_len

        history.append(best_len)

        # TODO: cooling schedule
        # How is T adjusted?

        if cfg.report_every and (it + 1) % cfg.report_every ==
0:
            print(f"[SA] iter={it+1:6d}  T={T:.4g}  cur=
{cur_len:.4f}  best={best_len:.4f}")

    return best_tour, best_len, history
```

You're in a great spot — your acceptance rule is correct *and* it's wired into the loop in the right place. Now SA is missing only one thing: **how T changes each iteration**.

What the cooling schedule is (conceptually)

Temperature **must go down over time** so the algorithm becomes less willing to accept worse moves.

Early on: explore (accept some worse moves)
Late: exploit (mostly accept only improvements)

So your update needs to make `T`:

- stay **positive**

- **decrease** steadily
- not drop to ~0 instantly

The “most standard” cooling rule you’ll likely use

Most SA skeletons for assignments use **geometric cooling**:

- Start: `T = cfg.t0`
- Each iteration: `T = T * cfg.alpha`

Where:

- `alpha` is a number slightly less than 1, like 0.99, 0.999, etc.

How to confirm your notebook expects this

Look in the config (`cfg`) definition cell for SA. You’re looking for names like:

- `alpha`
- `cooling_rate`
- `t_min`
- `schedule`

If you see `cfg.alpha` , that’s basically the notebook screaming “multiply by alpha.”

Where to place it in your code

Right under the `# TODO: cooling schedule` comment (exactly where you pointed). That way:

- acceptance uses the current T for this iteration
- then you cool for the next iteration

Two common “gotchas” to avoid

1) Don’t cool before acceptance

If you update `T` at the top of the loop, you’re effectively shifting the schedule and may cool too fast.

Your structure is correct: **cool after** evaluating/accepting.

2) Don’t let T hit 0

Even with `alpha < 1` , after many iterations `T` can become extremely small.
That’s usually fine, but if your assignment config uses huge `iters` , you may want a lower

bound like:

- `T = max(T, cfg.t_min)`
  only if `t_min` exists in cfg or the notebook hints at it.

Don't invent parameters—use what the notebook already provides.

Quick sanity check after you implement cooling

Use the existing print line:

```
[SA] ... T=...
```

You want to see:

- `T` decreasing over time
- not staying constant
- not becoming negative

Also: you should see early iterations improve "noisily," later improve "smoothly."

If you paste the **SA config cell** (the part where `cfg` is defined—just that chunk), I can tell you exactly which variable name to use for cooling (alpha vs something else) without giving you any "final answers," just matching the skeleton's intent.

def simulated_annealing_tsp(cities: List[Point], init_tour: Tour, cfg: SAConfig)

Cool — that line just shows the function signature. The key is what's **inside** `SAConfig` (that's where the notebook tells you the intended cooling rule).

Here's how to proceed *without me giving you "the answer" outright*:

What you should do next (2 minutes)

1. Scroll *up* until you find where `SAConfig` is defined. It will look like one of these:

- `@dataclass class SAConfig: ...`

- or a simple class with attributes
- or a dict-like config being created

2. In that definition, look for fields like:

- `t0`
- `alpha` / `cooling` / `cooling_rate`
- `iters`
- maybe `t_min` or `min_temp`

How to choose the cooling schedule once you see the fields

- If you see an `alpha` (or "cooling_rate") that's a number near 1:
  - That almost always means **geometric cooling**: multiply T by that factor each iteration.
- If you see something like `cooling="linear"` or a field like `t_end`:
  - Then it might be linear: subtract a small amount each iteration, or interpolate from `t0` to `t_end`.
- If you see **no cooling parameter at all** (rare):
  - The notebook might want a simple schedule based on iteration number (like `T = t0 / (1 + it)`), but usually they still provide a parameter.

"No spoilers" hint that still helps

In your code you already used `cfg.t0`, so the cooling line will almost certainly reference **another** `cfg.something` too. The skeleton is designed so you don't invent constants.

Paste just the `SAConfig` definition cell

If you paste the `SAConfig` class/dataclass block (just that), I'll tell you:

- which field is meant for cooling
- where to update `T`
- how to keep `T` valid

...without writing the final full code for you.

```
@dataclass
class SAConfig:
    iters: int = 20_000
    t0: float = 0.2
    alpha: float = 0.9995
    seed: int = 123
    report_every: int = 2000
```

Perfect — that `alpha` field is the giveaway. This config is set up for **geometric cooling**.

What you do in the cooling TODO

Each iteration, reduce temperature by multiplying by `alpha`:

- `T` starts at `cfg.t0`
- after each iteration: `T` becomes `T * cfg.alpha`

That matches the intent of `alpha = 0.9995` (a slow, gentle cool over 20,000 iterations).

Why this makes sense with your numbers

- After 2000 iters (one report interval), temperature should be lower than 0.2 but not near zero.
- By the end, it should be much smaller, so the algorithm becomes "pickier."

Quick sanity check you can run (no extra code needed)

You already print `T` every `report_every`. After you add the update, when you run SA you should see `T=` decrease at each report line.

One tiny caution

Keep the cooling update **after** the accept/update logic (right where your TODO is). That way the acceptance decision uses the "current" temperature for that step.

If you implement that and run it, tell me what your printed `T` values look like at iter 2000 / 4000 / 6000 — we can sanity-check cooling speed and make sure nothing weird is happening.

T = T * cfg.alpha
T = max(T, 1e-12)

Yep — that's a solid, safe cooling implementation.

- `T = T * cfg.alpha` matches the intended **geometric cooling** (since `alpha` is in the config).
- `T = max(T, 1e-12)` is a practical guard to prevent division-by-zero / underflow issues later. It won't meaningfully change behavior unless `T` gets *extremely* tiny.

Two quick checks to make sure everything's behaving:

1. When you run, your report lines should show **T decreasing** every 2000 iterations.
2. You should see early on it sometimes accepts worse moves (you'll notice `cur` can bounce), and later it mostly stabilizes.

If you paste one or two of the `[SA]` report lines you get (e.g., at iter 2000 and 20000), I'll help you interpret whether SA is behaving as expected.

```
A] iter=  2000  T=0.07356  cur=8.3800  best=6.0214
[SA] iter=  4000  T=0.02705  cur=6.5972  best=6.0214
[SA] iter=  6000  T=0.00995  cur=5.3139  best=5.3139
[SA] iter=  8000  T=0.003659  cur=5.0057  best=5.0057
[SA] iter= 10000  T=0.001346  cur=5.0057  best=5.0057
[SA] iter= 12000  T=0.000495  cur=5.0057  best=5.0057
[SA] iter= 14000  T=0.0001821  cur=5.0057  best=5.0057
[SA] iter= 16000  T=6.696e-05  cur=5.0057  best=5.0057
[SA] iter= 18000  T=2.463e-05  cur=5.0057  best=5.0057
[SA] iter= 20000  T=9.057e-06  cur=5.0057  best=5.0057
SA best length: 5.0057
SA improvement vs NN: 16.87%
```

This output looks **exactly like healthy SA behavior**.

What the lines are telling you

1) Cooling looks right

- T drops smoothly from 0.2 → 0.0736 (iter 2000) → … → ~9e-06 (iter 20000)
- That's consistent with `alpha=0.9995` over 20k steps. No sudden collapse, no weird increases.

2) The "explore then freeze" pattern is correct

- Early on:
  - `cur` is worse than `best` (e.g., iter 2000: cur 8.38 vs best 6.02)
  - That's normal: SA wanders while searching.
- Mid run:
  - You get improvements (best hits 5.3139 at 6000, then 5.0057 at 8000)
- Late run:
  - `cur == best` stays constant from 8000 onward
  - That means temperature is low enough that it's basically greedy / frozen. Totally expected.

Is it "too frozen too early"?

Not really. Freezing around 8k/20k is fine because you already found a strong solution and kept it.

If you *wanted* more exploration (not necessary unless the assignment asks you to tune):

- raise `t0` a bit, **or**
- make `alpha` closer to 1 (cool slower)

But for an intro SA assignment, your run looks great.

One quick optional debug check (to build intuition)

For like 50 iterations early in the run (e.g., it < 500), print a count of how many worse moves are accepted. You should see some. Then do the same near the end (e.g., last 500 iterations) and it should be near zero. This helps you "feel" SA.

If your SA section is done, the next intimidation hump is usually **GA order crossover (OX)**. If you want, paste the GA TODO stubs (function headers + comments), and I'll guide you

through implementing them with small notebook tests.

```
@dataclass
class GAConfig:
    pop_size: int = 200
    generations: int = 400
    tournament_k: int = 5
    crossover_rate: float = 0.9
    mutation_rate: float = 0.2
    elite_size: int = 5
    seed: int = 999
    report_every: int = 50

def tournament_select(pop: List[Tour], lengths: List[float], k:
int, rng: random.Random) -> Tour:
    # TODO: tournament selection (minimize length)
    raise NotImplementedError

def order_crossover_ox(parent1: Tour, parent2: Tour, rng:
random.Random) -> Tour:
    # TODO: order crossover (OX)
    raise NotImplementedError

def mutate_swap(tour: Tour, rng: random.Random) -> Tour:
    # TODO: swap mutation
    raise NotImplementedError

def genetic_algorithm_tsp(cities: List[Point],
init_seed_tours: List[Tour], cfg: GAConfig) -> Tuple[Tour,
float, List[float]]:
    rng = random.Random(cfg.seed)
    n = len(cities)

    pop: List[Tour] = []
    for t in init_seed_tours:
        assert is_valid_tour(t, n)
        pop.append(t[:])
```

```
    while len(pop) < cfg.pop_size:
        pop.append(random_tour(n, rng))

    lengths = [tour_length(cities, t) for t in pop]
    best_idx = min(range(len(pop)), key=lambda i:
lengths[i])
    best_tour = pop[best_idx][:]
    best_len = lengths[best_idx]
    history = [best_len]

    for gen in range(cfg.generations):
        elite_indices = sorted(range(len(pop)), key=lambda i:
lengths[i])[: cfg.elite_size]
        next_pop = [pop[i][:] for i in elite_indices]

        while len(next_pop) < cfg.pop_size:
            p1 = tournament_select(pop, lengths,
cfg.tournament_k, rng)
            p2 = tournament_select(pop, lengths,
cfg.tournament_k, rng)

            if rng.random() < cfg.crossover_rate:
                child = order_crossover_ox(p1, p2, rng)
            else:
                child = p1[:]

            if rng.random() < cfg.mutation_rate:
                child = mutate_swap(child, rng)

            next_pop.append(child)

        pop = next_pop
        lengths = [tour_length(cities, t) for t in pop]

        gen_best_idx = min(range(len(pop)), key=lambda i:
lengths[i])
        gen_best_len = lengths[gen_best_idx]
        if gen_best_len < best_len:
            best_len = gen_best_len
            best_tour = pop[gen_best_idx][:]
```

```
        history.append(best_len)

        if cfg.report_every and (gen + 1) % cfg.report_every
== 0:
            print(f"[GA] gen={gen+1:4d}  best={best_len:.4f}")

    assert is_valid_tour(best_tour, n)
    return best_tour, best_len, history
```

Great — GA is just 3 small building blocks. We'll do them in the safest order and you'll test each with tiny "toy tours" so it doesn't feel like a black box.

Order to implement (do this order)

1. `mutate_swap` (easiest)
2. `tournament_select`
3. `order_crossover_ox` (most fiddly)

---

1) `mutate_swap(tour, rng)`

Goal: return a *new* tour where two positions are swapped.

Steps

- Make a copy (so you don't mutate the input unexpectedly): `child = tour[:]`
- Pick two *different* indices `i, j` in `[0, len(tour)-1]`
  - easiest: `i, j = rng.sample(range(len(tour)), 2)`
- Swap: `child[i], child[j] = child[j], child[i]`
- Return `child`

Micro-test cell (add right under your function)

- Use `tour = [0,1,2,3,4,5]`
- Call mutation 5 times and check:
  - still same elements (permutation)
  - differs in exactly 2 positions (optional)

---

2) `tournament_select(pop, lengths, k, rng)`

Goal: choose one individual using tournament selection minimizing `lengths`.

Steps

- Randomly pick `k` indices from the population:
  - usually *with replacement is fine* (many simple GAs do that)
  - simplest: `[rng.randrange(len(pop)) for _ in range(k)]`
  - (or without replacement via `sample` if `k <= pop_size`)
- Among those indices, find the one with **smallest** `lengths[i]`
- Return a **copy** of that tour: `pop[best_i][:]`

Micro-test

- Make a tiny pop:
  - `pop = [[0],[1],[2],[3]]` (fake tours)
  - `lengths = [10, 5, 7, 1]`
  - With k=2, you'll often pick the "1" or "5" depending on sampled indices.
- If you set `k=len(pop)` it should *always* return the best one (length 1).

---

3) `order_crossover_ox(parent1, parent2, rng)`

This is the only tricky one. Here's the clean mental model:

OX in 5 steps

Let `n = len(parent1)`.

1. Create a child full of "empty slots":
   - e.g., `child = [-1] * n`
2. Choose two cut points `a < b` (indices):
   - Use `a, b = sorted(rng.sample(range(n), 2))`
   - Decide whether your slice is **inclusive/exclusive**. In Python it's easiest to treat it as `child[a:b]` (b is exclusive).
3. Copy the slice from `parent1` into the child:
   - `child[a:b] = parent1[a:b]`
4. Fill the remaining slots using `parent2` order:
   - Walk through `parent2` starting at `b`, wrapping around:

    - for each city in that order:
        - if the city is **not already in child**, place it into the next available empty slot in child (starting from `b`, wrapping around)
5. Return child.

What "starting at b and wrapping" means

- Build a scan order of indices: `b, b+1, ..., n-1, 0, 1, ..., b-1`
- That's the order you consider parent2's elements.
- Filling empty slots uses the same wraparound behavior.

Micro-test for OX (super important)

Use small parents so you can eyeball it:

- `p1 = [0,1,2,3,4,5,6,7]`
- `p2 = [3,7,5,1,6,0,2,4]`
  Run crossover a few times and check:
- child is a permutation (all cities exactly once)
- child's `a:b` segment matches `p1[a:b]`
- the other cities appear in the same relative order as in `p2` (starting from b wrap) excluding the copied ones

---

Common GA gotchas (so you don't waste an hour)

- **Forgetting to copy tours** and accidentally mutating pop members in-place.
- **OX producing duplicates** → usually because you didn't check "already in child" before inserting.
- **Slice endpoints** off by one → pick one convention (`a:b` exclusive) and stick to it.

Your next move

Implement **mutate_swap** first and paste it here (just that function). I'll "code review" it for correctness and permutation safety (without rewriting it into a full solution). Then we'll do tournament selection, then OX.

```
def mutate_swap(tour: Tour, rng: random.Random) -> Tour:
    # DONE: swap mutation
    child = tour[:]
    i = rng.randrange(0, len(tour))
    k = rng.randrange(0, len(tour))
    child[i], child[k] = child[k], child[i]
    return child
```

Nice — this is basically correct and safe (you copy first, then swap).

Two small tweaks to make it robust:

1) Ensure the two indices are different

Right now `i` and `k` can be the same, which means "mutation" sometimes does nothing. That's not *wrong*, but it weakens mutation rate in practice.

Easy pattern:

- pick `i`
- pick `k` until `k != i`
  (or use `rng.sample(range(len(tour)), 2)` which guarantees distinct)

2) Minor style nit (not required)

`rng.randrange(len(tour))` is enough; no need for the `0,`.

That's it — good.

If you want, paste your `tournament_select` next and I'll guide you to make sure it's minimizing length correctly and returning a safe copy.

```
# TODO: tournament selection (minimize length)
    k_indices = rng.sample(range(len(pop)), k)
    k_lengths = [lengths[i] for i in k_indices]
```

```
min_idx = min(range(k), key=lambda i: k_lengths[i])
return pop[k_indices[min_idx]][:]
```

This is really close, and the logic is correct: you sample `k` candidates, find the smallest length, return that tour.

A couple small improvements (mostly clarity / robustness):

1) You can avoid building `k_lengths`

Right now you do a two-step "length list then min over 0..k-1". Works fine, but you can directly pick the best sampled index:

- find `best_i` among `k_indices` using `lengths[i]` as the key
- return `pop[best_i][:]`

Same behavior, fewer moving parts.

2) Watch out for `k > len(pop)`

`rng.sample` requires `k <= len(pop)`. In your config, `k=5` and pop size is 200, so you're safe. But if someone changes config later, it'll crash. Optional guard:

- `k = min(k, len(pop))`

Not necessary for the assignment unless they test edge cases.

3) Returning a copy ✅

`pop[...][:]` is good — prevents accidental mutation of the population later.

So: **your implementation is correct** as-is.

Next up is the big one: `order_crossover_ox`. Paste your starter attempt (even if incomplete) or just the TODO cell, and I'll walk you through a clean way to implement it and test it with tiny parents.

idk where to start wiht the order_crossover

No worries — OX feels confusing until you picture it as **"copy a chunk from parent1, then pour in the remaining cities in parent2 order."** Here's a step-by-step recipe you can follow linearly.

What OX must guarantee

- Child is a **valid tour** (a permutation: every city exactly once)
- Child keeps a **contiguous slice** from `parent1`
- The remaining cities appear in the **same relative order as in** `parent2`

The simplest way to implement OX (recipe)

Assume `parent1` and `parent2` are lists of city IDs, length `n`.

Step 1: Pick cut points

Pick two different indices and sort them:

- `a < b`
  Treat the slice as Python slice `[a:b]` (b is exclusive). This avoids off-by-one pain.

Step 2: Create an "empty" child

Make a list of length `n` filled with a placeholder like `-1`.

Step 3: Copy the slice from parent1

Copy `parent1[a:b]` into `child[a:b]`.

Now the child has some real city IDs in the middle and `-1` elsewhere.

Step 4: Fill remaining slots using parent2 order starting from b (wrap around)

This is the heart of OX.

You need two "wraparound" sequences:

(A) The order you read cities from parent2

Start at index `b`, wrap around:

- indices: `b, b+1, ..., n-1, 0, 1, ..., b-1`
  Collect cities in that order.

(B) The order you fill positions in the child

Also start at index `b`, wrap around:

- fill positions in: `b, b+1, ..., n-1, 0, 1, ..., b-1`
  But only fill where `child[pos] == -1`.

Then:

- for each city from parent2 (in order A):
  - if city is **not already in child**:
    - put it into the next empty position (order B)

Step 5: Return child

A tiny example to make it click

Say:

- `parent1 = [0,1,2,3,4,5,6,7]`
- `parent2 = [3,7,5,1,6,0,2,4]`
- cut points `a=2, b=5` → slice is positions 2,3,4

Copy slice from parent1:

- child = `[-1, -1, 2, 3, 4, -1, -1, -1]`

Now read parent2 starting at b=5 (wrap):

- parent2 order: `[0,2,4,3,7,5,1,6]`

Skip cities already in child (2,3,4), fill the blanks in child starting at pos 5:

- fill positions: 5,6,7,0,1
- you'll place: 0→pos5, 7→pos6, 5→pos7, 1→pos0, 6→pos1

Result:

- child = `[1,6,2,3,4,0,7,5]` ✅ permutation ✅ slice preserved ✅ parent2 order preserved for the rest

---

How to code it without getting lost (structure, not the exact final code)

Inside `order_crossover_ox` you'll want these variables:

- `n = len(parent1)`
- `a, b = sorted(rng.sample(range(n), 2))`
- `child = [-1] * n`
- copy slice
- a set for fast membership checks:
  - `used = set(child[a:b])` (after slice copy)
- create the wraparound index list once:
  - `order = list(range(b, n)) + list(range(0, b))`
- then:
  - iterate parent2 in that `order`
  - iterate child positions in that `order` to fill

A clean approach is:

- build a list `p2_stream` of cities from parent2 in wrap order
- maintain a pointer `fill_idx` over `order` for child fill positions

Micro-test checklist (do this right after coding)

After generating a child, assert:

- `len(child) == n`
- `sorted(child) == sorted(parent1)` (permutation check)
- `child[a:b] == parent1[a:b]` (slice preserved)

If any assert fails, you instantly know where the bug is.

If you paste your notebook's helper function names (sometimes they have `is_valid_tour`) or you paste your *first attempt* at OX (even partial), I'll help you debug it step-by-step without writing the final full function for you.

```
parent1_len = len(parent1)
  child = [None] * parent1_len
  start = rng.randrange(parent1_len)
```

```
    end = rng.randrange(start, parent1_len)
    for i in range(start, end):
        child[i] = parent1[i]
```

Nice — you've already done the first key part: **allocate a child + copy a slice from parent1**. Now you just need the "OX fill" part.

A couple quick fixes/notes on what you have so far:

- Using `None` as the empty marker is fine (just be consistent).
- Your slice choice works, but it's easier/safer to use **two distinct cut points** and sort them:
  - this avoids weird cases where start==end and copies nothing.

Now here's the exact next steps to complete OX from where you are (guidance, not a full drop-in answer):

Step 1: Record what's already used

After copying the slice, make a set of cities already in the child:

- `used = set(x for x in child if x is not None)`

This makes "is this city already placed?" fast.

Step 2: Decide the wraparound order

OX specifically fills starting from `end` and wrapping around.

Make an index order like:

- indices `end, end+1, ..., n-1, 0, 1, ..., end-1`

This order will be used for:

1. reading cities from parent2 (starting at `end`)
2. filling empty child slots (starting at `end`)

So compute something like:

- `order = list(range(end, n)) + list(range(0, end))`

Step 3: Iterate parent2 in that order and fill child

You want:

- for each city in parent2 (in wrap order):
  - if city not in `used` :
    - put it in the next empty position in `child` (following the same wrap order)
    - add it to `used`

How to find the "next empty position"

You have two common approaches:

Approach A (pointer)

- keep a variable `fill_pos_idx = 0`
- while `child[order[fill_pos_idx]]` is not None: increment
- place city at that position

Approach B (precompute empties)

- `empty_positions = [idx for idx in order if child[idx] is None]`
- then fill empties in order with the filtered cities

Approach B is simpler to reason about, especially at first.

Step 4: Final checks (do this immediately)

After filling, verify:

- no `None` left
- child is a permutation (same set as parent1)

If your notebook has `is_valid_tour(child, n)` , call that.

Tiny improvements to your slice selection

Right now:

```python
start = rng.randrange(parent1_len)
```

Python Run

This allows `start == end`, which copies nothing.

Better pattern:

- pick two distinct indices and sort them:
  - `start, end = sorted(rng.sample(range(n), 2))`
    Then your slice is non-empty and consistent.

One important detail: slice endpoint convention

In Python, `range(start, end)` fills indices `start ... end-1`. That's good — just keep treating `end` as **exclusive** everywhere.

If you paste the next few lines you write for the "fill from parent2" part, I'll check it for the two classic bugs:

1. duplicates (forgetting to check `used`)
2. filling in the wrong order (not starting from `end` / not wrapping correctly)

```
parent1_len = len(parent1)
child = [None] * parent1_len
start = rng.randrange(parent1_len)
end = rng.randrange(start, parent1_len)
for i in range(start, end):
    child[i] = parent1[i]
used = child[:]
for i in range(parent1_len):
    if parent2[i] not in child:
        for j
        child[used.index(None)] = parent2[i]
```

used[used.index(None)] = parent2[i]

You're *very* close — you've got the right idea (copy slice, then fill with parent2 cities not already in child). The main issues are:

1. **You're not doing the wraparound order** that OX requires (starting from `end`).
2. `used = child[:]` isn't really "used"; it's just another copy of the same list.
3. `child.index(None)` works but is fragile/slow, and your code is halfway into a nested loop (`for j`) that you don't actually need.
4. Your fill should go into **the next empty slot in the wraparound fill order**, not "the first None from the front" every time.

Let's fix the structure (still guiding, not dumping a full final solution).

What to change, step by step

Step A — Make cut points clean

Your current selection can produce `start == end` (empty slice). Prefer:

- pick two distinct indices and sort them.

(You can keep your current way for now, but it's a common source of "child looks like parent2" bugs.)

Step B — Track used cities as a *set*

After slice copy:

- `used = set(x for x in child if x is not None)`

Now membership checks are fast and clear.

Step C — Create the OX wraparound index order

This is the key missing ingredient.

Make:

- `order = list(range(end, n)) + list(range(0, end))`

This order is how OX:

- reads from parent2 (starting at `end`)
- fills into the child (starting at `end`)

Step D — Precompute empty positions in that order

Instead of `index(None)` each time, do:

- `empty_positions = [idx for idx in order if child[idx] is None]`

Then keep a pointer:

- `pos_ptr = 0`

Step E — Iterate parent2 in wrap order and fill empties

Iterate the *cities* from parent2 in wrap order:

- for `idx` in `order`: `city = parent2[idx]`

If `city` not in `used`:

- place it in `child[ empty_positions[pos_ptr] ]`
- `pos_ptr += 1`
- add to `used`

That will exactly implement OX.

Why your current approach is "almost" OX but not quite

This part:

```
for i in range(parent1_len):
    if parent2[i] not in child:
        child[used.index(None)] = parent2[i]
```

fills from the **front** of the child, not from `end` with wraparound. That changes the operator (it becomes "order-based fill" but not classic OX as usually defined in TSP assignments).

Minimal debugging checks you should add right after

After building `child`, quickly validate:

- no `None` remains
- all cities are unique
- slice is preserved: `child[start:end] == parent1[start:end]`

(If your notebook has `is_valid_tour(child, n)` use that.)

If you want, paste your updated attempt after you add `order + empty_positions + pos_ptr`. I'll check it for the two most common remaining bugs: off-by-one on `end`, and filling the wrong positions.

Help me fix this one part at a time

Absolutely. We'll do **one tiny change at a time**.

Part 1 (first fix): make the cut points always valid + non-empty

Your current:

```
start = rng.randrange(n)
end = rng.randrange(start, n)
```

can give `start == end` (copies nothing), and it biases slice lengths.

Replace only those two lines with:

```python
```

Nothing else yet. Keep your slice-copy loop exactly as-is.

Quick check after this change

Add a temporary print once (or assert) to confirm it's non-empty:

- `assert start < end`

Do just this change first, run the notebook cell (Shift+Enter), and tell me:

- does it run without error?
- what values of `start` and `end` you see in one run (just one example)

Then we'll do Part 2.

part 2

Part 2: Track "used cities" correctly (as a set)

Right now you have:

Python Run

That doesn't actually help you track what's already placed.

Replace just that one line with:

Python Run

That's it for Part 2.

Why we do this

- `used` should mean "cities already in the child"
- A `set` makes membership checks (`city in used`) fast and clean

Quick check (optional but helpful)

Right after that line, add:

Python Run

Run the cell. If it works, say “ok” and we’ll do Part 3 (wraparound order).

okau

Great — onto **Part 3: create the wraparound order for OX**.

Right after you define `used`, add these two lines:

Python Run

That’s it for Part 3.

What this means

This produces indices:

- `end, end+1, ..., n-1, 0, 1, ..., end-1`
  which is the “start from end and wrap around” order OX uses.

Quick check (optional)

Add:

```
assert len(order) == parent1_len
assert len(set(order)) == parent1_len
```

Run the cell and tell me “ok” if it runs. Then we’ll do Part 4 (find empty positions in that order).

okay

Awesome. **Part 4: precompute the empty slots in the child (in OX fill order).**

Right after you make `order`, add:

```
empty_positions = [idx for idx in order if child[idx] is None]
pos_ptr = 0
```

That's all for Part 4.

Quick check (optional but good)

Add:

```python
```

Run the cell and tell me "ok". Then Part 5 will be the actual fill loop using parent2 in wrap order.

okay

Great. **Part 5: replace your whole "fill from parent2" block** with a clean OX fill using `order`, `empty_positions`, and `used`.

Delete/replace everything from:

```
for i in range(parent1_len):
```

Python Run

with this structure (fill in exactly as written; it's the standard OX loop pattern):

```
for idx in order:
    city = parent2[idx]
    if city not in used:
        child[empty_positions[pos_ptr]] = city
        used.add(city)
        pos_ptr += 1
```

That's it.

Part 5 sanity checks (add right after the loop)

```
assert None not in child
assert len(used) == parent1_len
```

Run it. If you get an error, paste the traceback line(s). If it runs, we'll do Part 6: final correctness checks (permutation + slice preserved).

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_2146/876836974.py in <cell line: 0>()
      2                 mutation_rate=0.2, elite_size=5, seed=999, report_every=50)
      3
----> 4 ga_best_tour, ga_best_len, ga_hist = genetic_algorithm_tsp(cities, init_seed_tours=[nn_tour], cfg=ga_cfg)
      5
```

```
    6 print(f"GA best length: {ga_best_len:.4f}")

1 frames
/tmp/ipykernel_2146/2045420550.py in mutate_swap(tour, rng)
     42 def mutate_swap(tour: Tour, rng: random.Random) -> Tour:
     43     # DONE: swap mutation
---> 44     child = tour[:]
     45     i = rng.randrange(0, len(tour))
     46     k = rng.randrange(0, len(tour))

TypeError: 'NoneType' object is not subscriptable
```

Good traceback — this error is **not** from OX directly. It says:

in `mutate_swap`: `child = tour[:]`
`TypeError: 'NoneType' object is not subscriptable`

So `mutate_swap` is being called with `tour = None`. That means **your crossover function is returning** `None` **sometimes** (or not returning at all).

Fix Part 5.5: make sure `order_crossover_ox` returns `child`

At the bottom of `order_crossover_ox`, you must have:

Python Run

If you forgot that (very common), Python returns `None` by default, and then mutation crashes exactly like this.

Also check: indentation

Make sure the `return child` is aligned with the function body (not inside a loop accidentally).

Quick verification step (tiny test cell)

Right after defining `order_crossover_ox`, run something like:

- call it once with two simple parents
- print the result and type

You should see a list-like tour, not `None`.

Do you already have `return child` at the end? If yes, paste the last ~10 lines of your `order_crossover_ox` function (including the return) and I'll spot the indentation/flow issue.

I forgot the return child statement, now i get this, looks like it works

A] gen= 100 best=5.9001
[GA] gen= 150 best=5.9001
[GA] gen= 200 best=5.9001
[GA] gen= 250 best=5.9001
[GA] gen= 300 best=5.9001
[GA] gen= 350 best=5.9001
[GA] gen= 400 best=5.9001
GA best length: 5.9001
GA improvement vs NN: 2.01%

Yep — that output means your GA loop is running end-to-end and producing valid tours. Nice.

A couple quick "is it behaving sensibly?" reads from your log:

- It **improves early** (since by gen 100 it's at 5.9001) and then **plateaus** through gen 400. That's normal for GA: once diversity drops, it can stall.
- The improvement vs NN is modest (2%). That can happen depending on the instance + parameters.

If you want to *validate correctness* (not "optimize"), do these quick checks:

1) Confirm crossover/mutation always produce valid tours

After crossover and after mutation (temporarily), add asserts:

- `assert is_valid_tour(child, n)`

If it never trips, your operators are producing permutations correctly.

2) Quick sanity: mutation sometimes changes the tour

Your `mutate_swap` can sometimes pick the same index twice → no change. It still works, but if you want stronger mutation:

- ensure the two indices differ (using `rng.sample(..., 2)`)

That often helps avoid stagnation.

3) Quick sanity: selection pressure is reasonable

`tournament_k=5` is fine. If you crank it up too high, you get fast convergence + plateau; too low, slow progress.

If your assignment wants you to compare SA vs GA: your results already show SA did better here (16.87% vs 2.01%). If you paste the comparison prompt/cell, I can guide you on what to write about the behavior (without giving you "answers," just what observations to look for and how to justify them).

This bit doesnt seem to do anything
def grade_check(nn_len: float, method_len: float, min_improvement_pct: float = 10.0, label: str = "Method") -> None:
    improvement = (nn_len - method_len) / nn_len * 100.0
    print(f"{label} length: {method_len:.4f} | Improvement vs NN: {improvement:.2f}%")
    assert improvement >= min_improvement_pct, (
        f"{label} improvement {improvement:.2f}% is below required {min_improvement_pct:.2f}%"

```
)
print(f"✅ {label} passed improvement threshold!")
```

It *does* do something, but only **when you call it** (and it's designed to "do nothing dramatic" unless you fail the threshold).

What it will do when called

1. Print the method length + improvement %
2. **Assert** the improvement is at least `min_improvement_pct`
   - If it's **below**, it will raise an `AssertionError` and stop the cell (that's the "grading" behavior)
   - If it's **above**, it prints the ✅ message

Why it might look like it "does nothing"

- If you never call `grade_check(...)`, defining the function won't print anything.
- If you call it and you're **passing**, you'll just see prints (no error).
- If you call it but assertions are disabled (rare in notebooks), it won't enforce the threshold.

The key thing in *your* results

Your GA improvement is **2.01%**, so if you call:

Python Run

...it should **fail** with an `AssertionError`. That would be "doing something."

Quick check: are assertions enabled?

Run this in a cell:

Python Run

- If you see an error: assertions are enabled (normal)

- If nothing happens: you're running Python with `-O` optimization (asserts disabled). In Jupyter that's uncommon, but possible in some setups.

If you want it to always enforce even when asserts are off

(only if allowed by the assignment)
Replace the `assert ...` with:

```
if improvement < min_improvement_pct:
    raise ValueError(...)
```

But I'd only do that if you confirm asserts are disabled.

If you paste the cell where you *think* you're using `grade_check`, I can point out exactly why it isn't triggering (most often: it's never called, or it's called with the wrong lengths/order).

I think its nevercalled then, but where would I call it so I can check out what it reports?

Call it **right after** you compute each method's best length (SA / GA), using the **NN length as the baseline**.

Where to put it

In the same cell where you already print results like:

- `SA best length: ...`
- `GA best length: ...`

Add these calls immediately after you have `nn_len`, `sa_best_len`, and `ga_best_len`.

Example placements (use your variable names)

If your notebook has something like:

```
nn_tour, nn_len = nearest_neighbor_tour(cities, start=0)
sa_best_tour, sa_best_len, sa_hist = simulated_annealing_tsp(cities, nn_tour, sa_
ga_best_tour, ga_best_len, ga_hist = genetic_algorithm_tsp(cities, init_seed_tour
```

Then right after each method finishes:

```
grade_check(nn_len, sa_best_len, min_improvement_pct=10.0, label="SA")
grade_check(nn_len, ga_best_len, min_improvement_pct=10.0, label="GA")
```

What you should expect with your current numbers

- **SA**: should pass (you got ~16.87%)
- **GA**: will likely **raise an AssertionError** (you got ~2.01%)

That's the point: it tells you whether you met the required improvement threshold.

One gotcha to avoid

Make sure the arguments are in this order:

- first: `nn_len`
- second: `method_len`

If you swap them, the "improvement" becomes negative and it'll always fail.

If you paste the variable names used for NN/SA/GA lengths in your notebook cell, I'll write the exact `grade_check(...)` lines using *your* names.

It says its not defined if I put it up there

Thought for a couple of seconds

That error means **Python hasn't seen the `grade_check` function yet** in the current kernel session (Jupyter only knows what you've *run*, not what's merely *above* on the page).

Fix it (pick one)

Option A: Run the cell that defines `grade_check`

1. Scroll to the cell that contains `def grade_check(...):`
2. Click it
3. Press **Shift+Enter**
4. Now re-run the results cell where you call `grade_check(...)`

This is the most "Jupyter-ish" fix.

Option B: Move the `grade_check` definition cell earlier

Cut/paste that entire function cell so it appears **above** the cell where you call it, then:

- **Kernel → Restart & Run All**

Option C: Redefine it in the same results cell (quick + dirty)

If you just want to test right now, paste the function definition at the top of the cell where you're calling it (not ideal long-term, but works).

Why this happens in notebooks

Jupyter executes in **execution order**, not visual order. If you ran the results cell before running the cell that defines `grade_check`, you'll get "not defined."

Quick "make it impossible to mess up" move

Do: **Kernel → Restart & Run All**
That guarantees everything gets defined in the correct order from top to bottom.

If you tell me what the cell number / section title is where `grade_check` lives (or paste the few lines above it), I can tell you the best place to position it in the notebook flow.

Oh looks like it did work:
NN: 6.0214 | SA: 5.0057 | GA: 5.9001

```
SA length: 5.0057 | Improvement vs NN: 16.87%
✅ SA passed improvement threshold!
GA length: 5.9001 | Improvement vs NN: 2.01%
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
/tmp/ipykernel_2146/3564383796.py in <cell line: 0>()
      7 # TODO
      8 grade_check(nn_len, sa_best_len, min_improvement_pct=10.0, label="SA")
----> 9 grade_check(nn_len, ga_best_len, min_improvement_pct=10.0, label="GA")

/tmp/ipykernel_2146/780773211.py in grade_check(nn_len, method_len, min_improvement_pct, label)
      2     improvement = (nn_len - method_len) / nn_len * 100.0
      3     print(f"{label} length: {method_len:.4f} | Improvement vs NN: {improvement:.2f}%")
----> 4     assert improvement >= min_improvement_pct, (
      5         f"{label} improvement {improvement:.2f}% is below required {min_improvement_pct:.2f}%"
      6     )

AssertionError: GA improvement 2.01% is below required 10.00%
```

Yep — that's exactly what it's supposed to do. ✅ SA passes, ❌ GA fails the "must improve by 10%" requirement.

Now the question is: how do you get GA to improve more **without me giving you a finished answer**. Here are the levers to try in a controlled way, one at a time.

First: confirm GA isn't "broken," just plateauing

Your GA log plateaued at 5.9001 for a long time, which usually means **not enough useful variation** (or too much selection pressure causing early convergence).

So we'll improve **exploration** and **mixing**.

Step 1: Fix the easy mutation issue (big impact, minimal change)

Your `mutate_swap` sometimes swaps the same index with itself → no mutation.

Change it so the two indices are always different (use `rng.sample`).

Then re-run and see if GA improvement increases.

Why this helps: your effective mutation rate is currently lower than 0.2.

Step 2: Add a "validity assert" while debugging (optional but helpful)

Inside the GA loop right after crossover and after mutation, temporarily:

- `assert is_valid_tour(child, n)`

If this ever fails, your operators are producing invalid tours and GA fitness becomes meaningless. (It *sounds* like you're valid, but this is a fast confidence boost.)

Step 3: Increase diversity (tuning knobs)

Try **only one change per run** so you know what mattered.

A) More mutation

- Increase `cfg.mutation_rate` a bit (e.g., 0.2 → 0.3 or 0.4)

B) Slightly weaker selection pressure

- Reduce `tournament_k` (e.g., 5 → 3)
  High k = quickly picks the same "good" parents = early convergence.

C) More generations

- Increase `generations` (e.g., 400 → 800)
  Only helps if you still have diversity; if you're flatlining, do A/B first.

D) Adjust elitism

- `elite_size=5` might be fine, but if convergence is too fast, try smaller (like 2–3).

Step 4: Make sure OX is doing what you think

A subtle OX mistake can still produce valid tours but weak mixing (child too similar to one parent).

Quick check (no need for deep math):

- Print a few example parents + child and visually see:
  - the slice from parent1 is preserved
  - the remaining order follows parent2

    If the child looks almost identical to parent1 most of the time, diversity will suffer.

What I would do in your situation (minimal, likely to pass)

1. Make `mutate_swap` guarantee distinct indices
2. Re-run GA with the same config
3. If still <10%:
   - lower `tournament_k` to 3 **or**
   - increase `mutation_rate` to ~0.3

     (try one first, not both)

If you paste your current `mutate_swap` after you modify it to guarantee distinct indices, I'll do a quick check. Then you run GA again and tell me the new "GA improvement vs NN" number.

```
def mutate_swap(tour: Tour, rng: random.Random) -> Tour:
    # DONE: swap mutation
    child = tour[:]
    i = rng.randrange(0, len(tour))
    k = rng.randrange(0, len(tour))
    child[i], child[k] = child[k], child[i]
    return child
```

You haven't made the "distinct indices" change yet — this is still the version that can pick the same index twice.

Change just one part (minimal edit)

Replace the two `randrange` lines with a single line that guarantees two different positions:

Python Run

Keep everything else the same.

So the function structure stays identical; only the index-picking changes.

Why this matters

With your current code, some fraction of "mutations" do nothing (when `i == k`). That makes GA stall more easily.

After you make that one-line change, re-run GA and paste the new:

- `GA best length`
- `GA improvement vs NN`

If it's still under 10%, we'll do exactly *one* config tweak next (I'd try `tournament_k = 3` first).

```
gen=  50  best=5.9001
[GA] gen= 100  best=5.9001
[GA] gen= 150  best=5.9001
[GA] gen= 200  best=5.9001
[GA] gen= 250  best=5.9001
[GA] gen= 300  best=5.9001
[GA] gen= 350  best=5.9001
[GA] gen= 400  best=5.90014
```

Good — that tells us mutation "doing something" wasn't the main bottleneck (or the GA is converging super fast anyway). It's **still plateauing at ~5.9001 from the start**, which usually points to **selection pressure + low diversity**.

Let's change **one thing** next, as promised.

Next single change: lower tournament size

In your `GAConfig`, change:

- `tournament_k: int = 5` → **3**

Why: `k=5` can be pretty aggressive with pop 200; it pushes the population to become similar quickly, which makes OX less able to create new combos.

What to do

1. Set `tournament_k = 3`
2. Re-run GA (same seed is fine for comparison)
3. Look for:
- best improving after gen 50 (instead of flatlining)
- final improvement closer to / above 10%

Paste the new GA report lines + final improvement.

If (and only if) it still flatlines

Then the next *single* knob I'd try is:

- bump `mutation_rate` from `0.2` → `0.35`

But do `k=3` first so we can attribute the effect.

still flatlines, no change with i change tournament_k to 3

Thought for 8s

Got it — if **changing** `tournament_k` **doesn't move the plateau at all**, that strongly suggests one of these is true:

1. **Your GA operators aren't creating meaningful new tours** (even if tours are valid), or
2. **Elitism is "locking in" the early best** and the rest of the population can't beat it, or
3. **Mutation is too weak** relative to the problem.

We'll fix this **one step at a time**, with the smallest diagnostic first.

Step 1: Verify OX is actually producing *new* children

Before tuning parameters, we should confirm crossover isn't accidentally returning something too close to `p1` every time.

Add this tiny debug snippet *once* (in a scratch cell), not inside the whole GA loop:

```
rng = random.Random(0)
p1 = pop[0]
p2 = pop[1]
child = order_crossover_ox(p1, p2, rng)

print("p1 head:", p1[:10])
print("p2 head:", p2[:10])
print("ch head:", child[:10])
print("child equals p1?", child == p1)
print("child equals p2?", child == p2)
print("valid?", is_valid_tour(child, len(cities)))
```

What you want to see:

- `child == p1` is usually **False**
- `child == p2` is usually **False**
- `valid?` is **True**

If `child == p1` is True most times, your OX is effectively not mixing.

Tell me what those three booleans print.

Step 2: If OX looks fine, reduce elitism (single change)

Set:

- `elite_size = 1` (from 5)

Why: elitism can cause fast takeover and stagnation, especially with strong selection/crossover.

Re-run GA and see if the best improves after gen 50.

Step 3: If still flat, increase mutation (single change)

Set:

- `mutation_rate = 0.5` (from 0.2)

Why: if crossover isn't generating improvements, mutation becomes the main exploration mechanism.

Re-run.

Step 4: If still flat, increase search budget (single change)

Pick ONE:

- `generations = 800` (from 400)

  or
- `pop_size = 400` (from 200)

Why I'm pushing the Step 1 debug first

Because if OX is quietly "degenerate" (valid but not mixing), no amount of parameter tuning will help much — it'll keep rediscovering the same basin.

So: run **Step 1** and paste these 3 lines of output:

- `child equals p1? ...`
- `child equals p2? ...`
- `valid? ...`

Then we'll do the next single change based on what you see.

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_2146/1871008903.py in <cell line: 0>()
      1 rng = random.Random(0)
----> 2 p1 = pop[0]
      3 p2 = pop[1]
      4 child = order_crossover_ox(p1, p2, rng)
      5

NameError: name 'pop' is not defined
```

Makes sense — `pop` only exists **inside** `genetic_algorithm_tsp`, so you can't access it from a separate cell.

Let's do the same test using tours you *do* have access to: `nn_tour` plus a random tour.

Step 1 debug (works outside GA)

Run this in a new cell:

```
import random

rng = random.Random(0)
n = len(cities)

p1 = nn_tour[:]                # your nearest-neighbor tour
p2 = random_tour(n, rng)       # a random tour from the provided helper

child = order_crossover_ox(p1, p2, rng)

print("child equals p1?", child == p1)
print("child equals p2?", child == p2)
print("valid?", is_valid_tour(child, n))
```

What to report back to me

Just those three printed lines.

- If `child equals p1?` is often True → OX isn't mixing (bug)
- If both are False and valid is True → OX is likely fine, and we move to tuning next (elitism/mutation)

Once you paste the three booleans, I'll tell you the next *single* change to try.

child equals p1? False
child equals p2? False
valid? True

Great — that means your OX is working (it's producing a real mixed, valid child). So the flatline is almost certainly **GA dynamics/tuning**, not a crossover bug.

Since you want this **one part at a time**, here's the next single change that most often breaks a plateau:

Step 2: Reduce elitism (single change)

Right now `elite_size = 5`. That can cause the population to "lock in" early and stop exploring.

Change in `GAConfig`:

- `elite_size: int = 5` → **1**

Leave everything else the same (including your mutation fix).

Why this helps

Elites are copied unchanged every generation. With 5 elites, a strong-but-not-great solution can dominate quickly, and crossover keeps recombining similar parents → no new best.

What to look for

Re-run GA and watch:

- Does `best=` continue improving after gen 50/100?
- Does final improvement move toward 10%?

Paste the new GA "best length" and improvement.

If it *still* flatlines after this, the next single knob we'll touch is **mutation_rate** (jump it to ~0.5). But do `elite_size=1` first.

changing elite_size did nothing, I even changed it to 10 and still nothing. So 1 does not effect the best

That's a really useful clue: if `elite_size` changes don't change anything (even to 10), it usually means the GA's **best tour is being found immediately (gen 0/1)** and then **nothing in the search operators is beating it** under the current budget. So we need **more exploration power**.

Next single change (and the most likely to move the needle):

Step 3: Increase mutation rate a lot

Set:

- `mutation_rate = 0.6` (from 0.2)

Leave everything else alone (keep your fixed `mutate_swap` with distinct indices).

Why such a big jump?

A swap mutation is a *small* change. If crossover isn't producing improvements beyond ~5.9001, you need mutation to kick the population into new basins.

What to look for

- You should start seeing `best=` sometimes improve later (not necessarily monotonic every report, but it should occasionally drop).

- If it gets worse/noisy early, that's okay; elites (even size 1) will preserve the best-so-far.

Run it and paste:

- final `GA best length`
- improvement %
- (and whether `best=` ever changed after gen 50)

If mutation_rate=0.6 still doesn't move it

Then we do the next *single* change: **generations = 2000** (increase search budget).
But try the mutation jump first.

Also, quick question (no need to answer if you don't want): does your assignment *require* GA to hit 10%, or is that threshold only meant for SA? Your `grade_check` is being applied to GA too, but sometimes the rubric only requires one method to clear it.

Part 7: Optional Task
Try different crossover and/or mutation operations in GA and see if you can improve the performance. Document your effort in this section.

Perfect — that means you *don't* have to force the provided GA to hit 10%. You can keep your correct OX+swap GA as the "base", then **experiment** with alternative operators and report what happened. The `grade_check` failing for GA is basically a hint that "baseline GA may not meet 10%, now try improvements."

Here's a clean, student-friendly way to proceed (still guidance, not dropping final code).

What to write/do for Part 7

You want to show:

1. what baseline GA achieved (yours: ~2%)
2. what you tried (operator changes)

3. what changed in results (better / worse / same)
4. a brief explanation why

High-impact operator tweaks you can try (one at a time)

Option A: Stronger mutation — "2-opt mutation" (best bet for TSP)

Swap mutation is tiny. For TSP, a **2-opt** move is a much more meaningful local improvement operator.

Idea: with some probability, replace `mutate_swap` with a `mutate_two_opt` that:

- picks two indices `i<j`
- reverses the segment between them
- returns the tour

Why it helps: 2-opt directly targets crossing edges, which often yields big improvements.

How to document:

- "Replaced swap mutation with 2-opt mutation (segment reversal). This increased GA improvement from X% to Y%."

Option B: Mix mutations (exploration + local improvement)

Instead of only one mutation:

- with 50% chance do swap
- with 50% chance do 2-opt

This often keeps diversity while still allowing strong steps.

Option C: Alternative crossover — "PMX" (Partially Mapped Crossover)

OX is fine, but PMX sometimes performs better depending on instance.

PMX idea (high level):

- copy a slice from parent1
- create a mapping for swapped cities between parents in the slice
- fill remaining positions using parent2 but resolve conflicts via the mapping

This is more complex than 2-opt mutation, so I'd try mutation first.

Option D: Add "local search after crossover" (memetic GA)

After you create `child`, run **a few 2-opt improvements** on it (like 5–20 attempts).

This often boosts performance a lot, but increases runtime.

A simple experiment plan (easy to write up)

Do 3 runs (same seed) and record best length / improvement:

1. **Baseline:** OX + swap mutation (your current)
2. **Mutation upgrade:** OX + 2-opt mutation
3. **Hybrid:** OX + (swap or 2-opt randomly)

Then in Part 7, write a short paragraph:

- what you changed
- what result you observed
- why you think it happened (2-opt is better aligned with TSP structure)

Quick tip about the `grade_check`

For Part 7, you can either:

- lower the required threshold for GA (e.g., 0% or 2%), **or**
- don't use `grade_check` for GA at all, just print comparisons

Because Part 7 is "optional experimentation," not a hard pass/fail.

If you want to try the most likely improvement first

Do **2-opt mutation**. You already have a function for SA neighbor generation:

`random_two_opt_neighbor(...)`.

That's great because you can reuse it conceptually:

- mutation can just call that neighbor function on the tour.

If you paste the definition/signature of `random_two_opt_neighbor` from the notebook (just that function), I'll guide you on how to adapt it into a mutation operator cleanly (without me writing the whole final GA for you).

I want to focus on setting up the report then Ill go back to experiementing. What does " which, SA or GA , showed more variability?

"More variability" means: **which method's results bounce around more while it's running** (and/or across different runs with different random seeds).

There are two common ways your assignment might mean it:

1) Variability during a single run (most likely)

Look at the *trajectory* as the algorithm runs.

SA: what variability looks like

- `cur_len` can go **up and down** because SA sometimes accepts worse moves.
- `best_len` usually only goes down (or stays flat), but may improve in bursts.

In your log:

- SA's `cur` changes a lot early (8.38 → 6.59 → 5.31 → 5.01), then stabilizes. That's variability.

GA: what variability looks like

- The printed value is `best=` (best-so-far), which is **non-increasing** by design. So it will often look *less variable*, even if the population is changing.

In your log:

- GA `best=` flatlined at 5.9001 almost immediately → **low variability** in best-so-far.

So using your printed outputs: SA showed more variability.

Important nuance: If you printed GA's **average population length** each generation, that would show variability; but your report prints only best-so-far.

2) Variability across multiple runs (seeds)

If you run SA or GA multiple times with different seeds, variability means:

- does the final best length vary a lot from run to run?

You *can* mention this as “we didn’t test multiple seeds” if you didn’t.

How to answer in your report (using your current evidence)

- “SA showed more variability during the run because the current solution length changed substantially early on due to probabilistic acceptance of worse moves, then stabilized as temperature cooled.”
- “GA’s reported metric (best-so-far) showed little variability because it plateaued quickly and best-so-far is monotonic.”

If your assignment has a plot of `history` for both, you can say:

- variability = how “jagged” the curve is / how often it changes.

If you paste the exact sentence/question from the report prompt, I can help you phrase the explanation tightly (still in your own words, not writing the whole report).

Thats what I thought it was. Next question "Did SA escape the local minima?" what is the minima here?

In this assignment, a “local minimum” means:

A tour where every *small neighbor move* you allow (here, 2-opt) fails to improve the length — i.e., if you did greedy hill-climbing with 2-opt, you’d get stuck there.

So the “minimum” is not a specific number like 5.0 by definition; it’s **a point in the search where your neighbor operator can’t find a better move**.

What does “escape” mean?

It means SA temporarily accepts a **worse** move (goes uphill) so it can later find a **better** tour that was unreachable by only taking improvements.

Concretely, SA "escapes a local minimum" if:

- it reaches a state where no improving 2-opt move is taken for a while, **then**
- it accepts some worse moves, **then**
- later finds a new best (best length decreases again).

In your run: did it escape?

From your log alone we can't *prove* it, because you're only printing every 2000 iterations and you're printing both `cur` and `best`. But the pattern is suggestive:

- At iter 2000: `cur=8.38`, `best=6.02`
- By iter 6000: `best=5.3139`
- By iter 8000: `best=5.0057`

That indicates it **kept improving past the NN baseline**, and the fact that `cur` was sometimes worse than `best` earlier is consistent with accepting uphill moves.

How to answer the question safely in a report

- **Define** local minimum: "a tour that cannot be improved by the allowed neighbor move (2-opt) without first taking a worse step."
- Then explain SA's mechanism: "SA can accept worse moves early due to temperature, which can allow it to move out of local minima."
- For your specific run, you can say something like:
  - "SA likely escaped local minima early (temperature still high) because it continued finding better best lengths over time rather than plateauing immediately."

If you want a simple way to *verify* (optional)

Track a counter like "how many consecutive iterations without improving `best`," and observe whether improvements happen after long flat stretches while T is still > 0. Or log whether any `delta > 0` moves were accepted early.

If you paste the exact wording of that prompt section, I can help you tailor the explanation to what they expect (e.g., whether they want evidence from the printed logs or just conceptual understanding).

Yes, current distance is reported as longer than best as some intervals suggesting the algorithm explored less promising paths at times. The best report continues to improve for sometime before it converges suggesting it did find a better path by exploring. This is consistent with temperature (T) decreasing in later iterations to reduce exploring.

That reasoning is on the right track. A couple tweaks will make it tighter and more "SA-specific" (and avoid overclaiming).

Key points to strengthen:

- Make "local minimum" explicit (w.r.t. **2-opt neighbors**).
- Tie "cur > best" to **accepting uphill moves** (not just "explored").
- Use the concrete iterations where best improved (6000, 8000) as evidence.

Here's a cleaned-up version you can adapt into your own words:

- **Local minimum meaning:** a tour where no improving 2-opt move is available, so a purely greedy 2-opt search would stall.
- **Evidence of escape behavior:** In the run, `cur_len` is sometimes larger than `best_len` (e.g., at 2000 and 4000 iterations), which indicates the algorithm did not always stay at the best-found solution and likely accepted some uphill moves. Meanwhile, `best_len` continued to drop later (notably improving around 6000 and again around 8000 iterations) before flattening out.
- **Why that fits SA:** This matches SA's design: early higher temperature allows occasional acceptance of worse moves (helping escape local minima), and as `T` decreases the search becomes more conservative and eventually converges (best stops improving).

If you want one extra sentence that's honest:

- "From the coarse reporting alone we can't directly prove *which* uphill moves caused each improvement, but the pattern is consistent with SA escaping local minima early

and then cooling into exploitation."

If you paste the paragraph where you'll place this in your report, I can help you make it flow with the surrounding sentences (still keeping it your voice).

Never suggest to write anything in my voice again, if thats what I want I will state it. Remember this request

Updated saved memory

Thought for a couple of seconds

Got it — I won't suggest wording "in your voice" or offer rewrite-style phrasing unless you explicitly ask for it.

Is there anything misaligned in this:
Yes, current distance is reported as longer than best as some intervals suggesting the algorithm explored less promising paths at times. The best report continues to improve for sometime before it converges suggesting it did find a better path by exploring. This is consistent with temperature (T) decreasing in later iterations to reduce exploring.

A few small misalignments / overstatements to watch for:

- **"cur > best" doesn't directly prove exploration** on its own. It proves the algorithm is currently at a solution worse than the best-so-far (which is expected even in many algorithms). For SA specifically, it's *consistent with* accepting uphill moves, but your sentence reads a bit like it's definitive evidence.
- **"less promising paths"** is a value judgment. More precise is "higher-cost tours" or "worse objective values."

- **"best report continues to improve"**: true in your run, but it's better to anchor it to what you observed (best improved until ~8000, then plateaued).
- **Temperature phrasing:** "reduce exploring" is basically right, but more accurate is: lower `T` reduces the probability of accepting worse moves, so behavior becomes more greedy/convergent.

If you want a checklist to ensure alignment with SA mechanics:

- Define local minimum relative to the neighbor operator (2-opt).
- Say "consistent with" rather than "suggesting/proving," unless you actually logged accepted uphill moves (delta > 0 accepted).
- Tie convergence to decreasing acceptance probability as `T` drops.

Did GA lose diversity?

I think it did due to a low population count and only a few iterations(generations) it flatlines very quickly

Your conclusion ("it lost diversity / converged early") is consistent with the **observed behavior**: the GA best-so-far plateaus almost immediately and never improves after that.

Two things to tighten conceptually (so it matches what "diversity" actually means):

- **Flatlining best-so-far is not direct evidence of low diversity.** It's evidence of "no better individual was found." Loss of diversity is a common cause, but you didn't measure diversity.
- **Population count and generations:** your `pop_size=200` is not obviously "low" in general; whether it's low depends on the problem size (number of cities). Also, 400 generations isn't "few" on its own. The stronger point is "it plateaued very early *despite* many generations," which suggests premature convergence/stagnation.

What you *can* say safely (aligned with your logs):

- GA likely **stagnated / prematurely converged**, since best length reached ~5.9001 by early generations and didn't improve through gen 400.

If you want a quick way to **support** the “lost diversity” claim (optional, not required):

- Log and plot **population statistics** per generation:
  - average length
  - standard deviation of lengths
    If std dev collapses quickly, that’s real evidence diversity dropped.
    (Or compute simple genotype diversity like average Hamming distance between tours, but length-std-dev is easiest.)

Also note a nuance: even with diversity, GA can stagnate if the operators are weak for TSP (swap mutation is mild), so “low diversity” isn’t the only explanation.

If you tell me roughly how many cities are in your TSP instance (just `n = len(cities)`), I can help you judge whether pop_size=200 is small/medium/large in context—without turning it into “write this sentence.”

```
Point = Tuple[float, float]
Tour = List[int]

def make_cities(n: int = 40, seed: int = 7) -> List[Point]:
    # Reproducible 2D city coordinates in [0, 1]x[0, 1].
    rng = random.Random(seed)
    return [(rng.random(), rng.random()) for _ in range(n)]

def dist(a: Point, b: Point) -> float:
    dx = a[0] - b[0]
    dy = a[1] - b[1]
    return math.hypot(dx, dy)

def tour_length(cities: List[Point], tour: Tour) -> float:
    # Total length of a closed tour.
    n = len(tour)
    total = 0.0
    for i in range(n):
        total += dist(cities[tour[i]], cities[tour[(i + 1) % n]])
    return total
```

```
def is_valid_tour(tour: Tour, n: int) -> bool:
    return len(tour) == n and set(tour) == set(range(n))

def random_tour(n: int, rng: random.Random) -> Tour:
    t = list(range(n))
    rng.shuffle(t)
    return t

def nearest_neighbor_tour(cities: List[Point], start: int = 0) -
> Tour:
    # Deterministic baseline: greedy nearest neighbor.
    n = len(cities)
    unvisited = set(range(n))
    tour = [start]
    unvisited.remove(start)
    cur = start
    while unvisited:
        nxt = min(unvisited, key=lambda j: dist(cities[cur],
cities[j]))
        tour.append(nxt)
        unvisited.remove(nxt)
        cur = nxt
    return tour
```

With **n = 40 cities**, `pop_size = 200` is **not obviously low**. That's a pretty reasonable population size for a small TSP instance.

So your "lost diversity because low population count / few generations" explanation is a bit misaligned with the numbers you actually used.

What *is* aligned with what you observed:

- The GA **stagnated early** (best-so-far flatlined almost immediately and stayed there through 400 generations). That is consistent with *premature convergence* or *operators not generating improvements*.

More accurate possible causes (that match your setup):

- **Mutation operator strength:** swap mutation is a very small perturbation for TSP; it often struggles to make the bigger structural improvements that 2-opt can.

- **Crossover + selection dynamics:** even with pop 200, tournament selection + elitism can reduce *effective* diversity quickly, leading to stagnation.
- **Metric you printed:** you only printed **best-so-far**. That can flatline even if the population is still diverse; you didn't measure diversity directly.

If you want to support "lost diversity" with actual evidence (quick to compute):

- Track **std dev of** `lengths` per generation (if it collapses, that's diversity loss in fitness space).
- Track **mean length** too (if mean stops improving and std dev shrinks, that's strong stagnation evidence).

So: you can keep the "likely lost diversity / converged early" idea, but don't pin it on "low pop / few generations" for n=40 unless you add a measurement showing diversity dropped.

I got improvement out of GA finally! I was editing the wrong place in the notebook

Nice — that's a classic notebook trap.

To make sure you can *trust* the new GA improvement (and not get bitten by "ran the wrong cell order" again), do these quick checks:

1. **Restart & Run All**

- Kernel → Restart & Run All
- This guarantees the code you edited is the code being executed.

2. Confirm the right function is being used

- In the cell where you call `genetic_algorithm_tsp(...)`, make sure it's using the config values you think (pop size, generations, mutation rate, etc.).
- Re-print the config object right before running GA.

3. Re-run `grade_check` for GA

- `grade_check(nn_len, ga_best_len, min_improvement_pct=10.0, label="GA")`
- If it passes after Restart & Run All, you're solid.

If you paste your new GA best length + improvement %, I can help you interpret whether the behavior looks reasonable (e.g., steady improvement vs sudden jump) and what to mention in your report without rewriting anything for you.

```
[GA] gen=  50  best=5.2744
[GA] gen= 100  best=4.9849
[GA] gen= 150  best=4.9849
[GA] gen= 200  best=4.9646
[GA] gen= 250  best=4.9646
[GA] gen= 300  best=4.9646
[GA] gen= 350  best=4.9646
[GA] gen= 400  best=4.9646
```

it still converges pretty early

Yeah — but this is a *healthy* kind of "early convergence" now, because it's converging to a much better solution.

From your log:

- Big improvements happen by gen 100 (5.2744 → 4.9849)
- Small improvement by gen 200 (4.9849 → 4.9646)
- Then it plateaus through gen 400

That's a standard GA pattern: fast gains early, then diminishing returns.

How to interpret it for your report

- GA made most progress in the early generations, then stagnated (no better tours discovered after ~200).
- This suggests either:
  - the population lost useful diversity (premature convergence), **or**
  - the operators/parameters weren't strong enough to keep exploring new basins after that point.

You can say “converged early” based on the plateau, but be careful: plateau ≠ guaranteed diversity loss unless you measured it. It *is* clear stagnation.

If you want one simple way to back up “lost diversity” (optional, quick)

During the GA loop, record each generation:

- `avg_len = sum(lengths)/len(lengths)`
- `std_len = (mean of squares - mean^2)**0.5`

If `std_len` shrinks a lot early and stays small, that’s evidence of diversity collapse (in fitness space).

If you wanted to push it further (not necessary unless you’re experimenting)

Single-parameter levers that delay plateau:

- increase `mutation_rate`
- reduce `tournament_k`
- reduce `elite_size`
- run more generations (only helps if diversity remains)

But for reporting, your current curve already shows the story clearly: **rapid improvement then plateau**.

If you tell me your NN length (6.0214 earlier?) and this final GA best (4.9646), I can help you compute/verify the improvement percentage you should report.